

Отчет N3

ClickHouse: Engine = Null, ReplicatedMergeTree, Materialized View и проверка репликации логов

Параметр	Значение
Вариант	2, задание со звездочкой
Стенд	Docker Compose: ZooKeeper + две реплики ClickHouse
Реплики	clickhouse01, clickhouse02
База данных	hw
Основная проверка	одинаковые строки доступны на обеих репликах; system.replicas: is_readonly=0, absolute_delay=0, queue_size=0

Цель работы: настроить входную таблицу для логов с движком Engine = Null, целевую реплицируемую таблицу такой же структуры с материализованным полем для идентификации реплики, а также Materialized View для переноса логов из Null-таблицы в ReplicatedMergeTree. Дополнительно поднята вторая реплика и проверена репликация логов.

1. Секции конфигурации

Для стенда использованы две реплики ClickHouse и ZooKeeper. Ниже приведены секции, которые требуются для сдачи: remote_servers, zookeeper и macros.

1.1 docker-compose.yml

```
services:
  zookeeper:
    image: zookeeper:3.9
    container_name: zookeeper
    hostname: zookeeper
    ports:
      - "2181:2181"

  clickhouse01:
    image: clickhouse/clickhouse-server:24.8
    container_name: clickhouse01
    hostname: clickhouse01
    ports:
      - "9001:9000"
      - "8124:8123"
    environment:
      - CLICKHOUSE_SKIP_USER_SETUP=1
    volumes:
      - ./clickhouse01/config.xml:/etc/clickhouse-server/config.d/cluster.xml
    depends_on:
      - zookeeper

  clickhouse02:
    image: clickhouse/clickhouse-server:24.8
    container_name: clickhouse02
    hostname: clickhouse02
    ports:
      - "9002:9000"
      - "8125:8123"
    environment:
      - CLICKHOUSE_SKIP_USER_SETUP=1
    volumes:
      - ./clickhouse02/config.xml:/etc/clickhouse-server/config.d/cluster.xml
    depends_on:
      - zookeeper
```

1.2 Конфигурация первой реплики clickhouse01

```
<clickhouse>
  <listen_host>0.0.0.0</listen_host>

  <remote_servers>
    <logs_cluster>
```

```

        <shard>
          <internal_replication>true</internal_replication>
          <replica>
            <host>clickhouse01</host>
            <port>9000</port>
          </replica>
          <replica>
            <host>clickhouse02</host>
            <port>9000</port>
          </replica>
        </shard>
      </logs_cluster>
    </remote_servers>

    <zookeeper>
      <node>
        <host>zookeeper</host>
        <port>2181</port>
      </node>
    </zookeeper>

    <macros>
      <cluster>logs_cluster</cluster>
      <shard>01</shard>
      <replica>clickhouse01</replica>
    </macros>
  </clickhouse>

```

1.3 Конфигурация второй реплики clickhouse02

```

<clickhouse>
  <listen_host>0.0.0.0</listen_host>

  <remote_servers>
    <logs_cluster>
      <shard>
        <internal_replication>true</internal_replication>
        <replica>
          <host>clickhouse01</host>
          <port>9000</port>
        </replica>
        <replica>
          <host>clickhouse02</host>
          <port>9000</port>
        </replica>
      </shard>
    </logs_cluster>
  </remote_servers>

  <zookeeper>
    <node>
      <host>zookeeper</host>
      <port>2181</port>
    </node>
  </zookeeper>

  <macros>
    <cluster>logs_cluster</cluster>
    <shard>01</shard>
    <replica>clickhouse02</replica>
  </macros>
</clickhouse>

```

2. SQL-команды CREATE TABLE и CREATE MATERIALIZED VIEW

База данных создана на обеих репликах:

```
CREATE DATABASE IF NOT EXISTS hw;
```

2.1 Таблица-источник с Engine = Null

Таблица принимает вставки логов. Данные в ней не хранятся, но вставка используется как источник для Materialized View.

```
CREATE TABLE hw.logs_null
(
    event_time DateTime,
    level LowCardinality(String),
    message String,
    user_id UInt64
)
ENGINE = Null;
```

2.2 Реплицируемая таблица с материализованным полем replica_name

Поле replica_name вычисляется через getMacro('replica') и показывает, какая реплика обработала вставку.

```
CREATE TABLE hw.logs_replicated
(
    event_time DateTime,
    level LowCardinality(String),
    message String,
    user_id UInt64,
    replica_name String MATERIALIZED getMacro('replica')
)
ENGINE = ReplicatedMergeTree(
    '/clickhouse/tables/{shard}/hw/logs_replicated',
    '{replica}'
)
ORDER BY (event_time, user_id);
```

2.3 Materialized View для переноса логов

```
CREATE MATERIALIZED VIEW hw.logs_mv
TO hw.logs_replicated
AS
SELECT
    event_time,
    level,
    message,
    user_id
FROM hw.logs_null;
```

2.4 Тестовые вставки

```
-- Вставка через первую реплику
INSERT INTO hw.logs_null
(event_time, level, message, user_id)
VALUES
    (now(), 'INFO', 'log from replica 01', 1001),
    (now(), 'ERROR', 'second log from replica 01', 1002);

-- Вставка через вторую реплику
INSERT INTO hw.logs_null
(event_time, level, message, user_id)
VALUES
    (now(), 'WARN', 'log from replica 02', 2001);
```

3. Проверка результата

На обеих репликах выполнен SELECT из таблицы hw.logs_replicated. Результат показывает одинаковый набор логов, включая записи, созданные через разные реплики.

3.1 Результат SELECT на clickhouse01

```
yc-user@compute-vm-2-4-30-ssd-1776951461260:/tmp/ch_star$ docker exec -it clickhouse01 clickhouse-client --query "
SELECT
  level,
  message,
  user_id,
  replica_name
FROM hw.logs_replicated
ORDER BY event_time
FORMAT PrettyCompact
"
```

	level	message	user_id	replica_name
1.	INFO	log from replica 01	1001	clickhouse01
2.	ERROR	second log from replica 01	1002	clickhouse01

	level	message	user_id	replica_name
3.	WARN	log from replica 02	2001	clickhouse02

Скриншот 1. На первой реплике видны логи от clickhouse01 и clickhouse02.

3.2 Результат SELECT на clickhouse02

```
yc-user@compute-vm-2-4-30-ssd-1776951461260:/tmp/ch_star$ docker exec -it clickhouse02 clickhouse-client --query "
SELECT
  level,
  message,
  user_id,
  replica_name
FROM hw.logs_replicated
ORDER BY event_time
FORMAT PrettyCompact
"
```

	level	message	user_id	replica_name
1.	INFO	log from replica 01	1001	clickhouse01
2.	ERROR	second log from replica 01	1002	clickhouse01

	level	message	user_id	replica_name
3.	WARN	log from replica 02	2001	clickhouse02

Скриншот 2. На второй реплике виден тот же набор логов, что подтверждает репликацию данных.

3.3 Проверка system.replicas

Дополнительно проверена системная таблица system.replicas. Для таблицы hw.logs_replicated видно: is_readonly = 0, absolute_delay = 0, queue_size = 0.

```
yc-user@compute-vm-2-4-30-ssd-1776951461260:/tmp/ch_star$ docker exec -it clickhouse01 clickhouse-client --query "
SELECT
  database,
  table,
  replica_name,
  is_leader,
  is_readonly,
  absolute_delay,
  queue_size
FROM system.replicas
WHERE database = 'hw'
FORMAT PrettyCompact
"
```

	database	table	replica_name	is_leader	is_readonly	absolute_delay	queue_size
1.	hw	logs_replicated	clickhouse01	1	0	0	0

Скриншот 3. Состояние репликации для таблицы logs_replicated.

4. Вывод

Задание со звездочкой выполнено. Создана таблица логов с Engine = Null, создана реплицируемая таблица ReplicatedMergeTree с материализованным полем replica_name, настроено Materialized View для переноса логов в реплицируемую таблицу. Вторая реплика поднята, а репликация подтверждена одинаковыми результатами SELECT на clickhouse01 и clickhouse02, а также состоянием system.replicas без задержки и очереди.